

Multi-Agent Code Refactoring Benchmark Evaluation Report

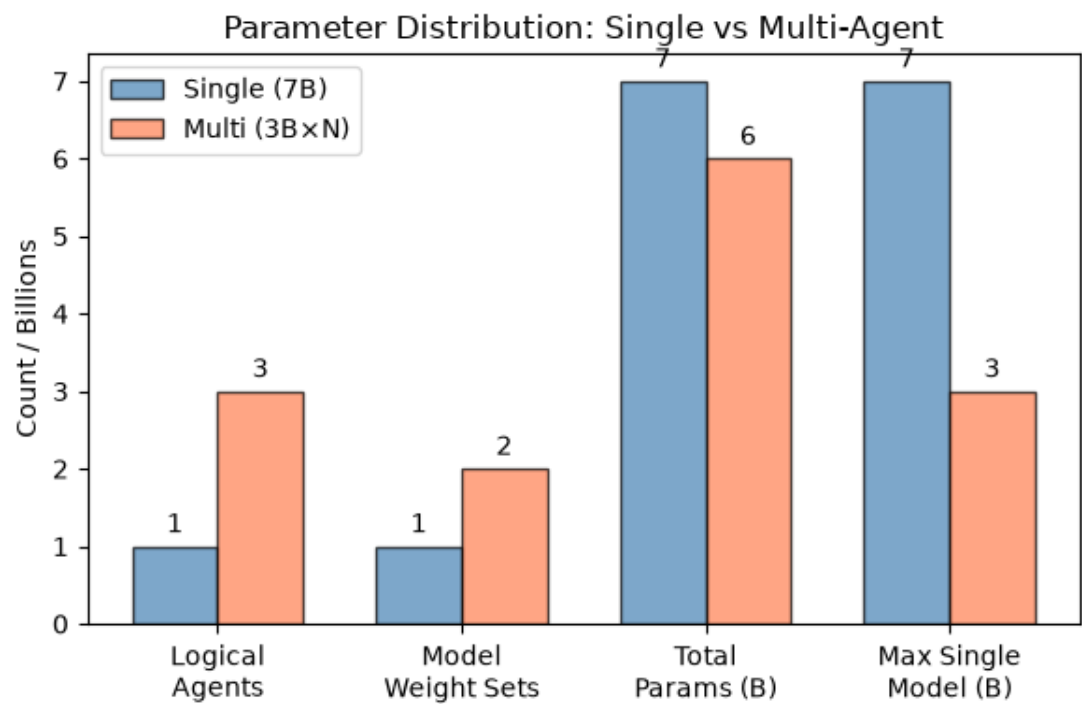
Overview & System Architecture

This report presents the empirical results of evaluating the multi-agent code refactoring framework against the monolithic single-model baseline. The data gathered from the benchmark execution provides detailed findings regarding algorithmic constraints, refactoring performance, and computational efficiency.

The researchers enhanced the code refactoring process by utilizing a multi-agent framework. Instead of a monolithic single-pass generation, the simulated agents are distributed into specialized roles—Planner, Generator, and Judge—operating with smaller 3B parameter models. This bounds the search and generation space, evaluates candidate outputs using deterministic compilation checks, and repairs errors iteratively. This makes the agents explore only high-quality partial solutions where the search space shrinks drastically, reducing wasted evaluations.

1. System Features and Algorithmic Constraints

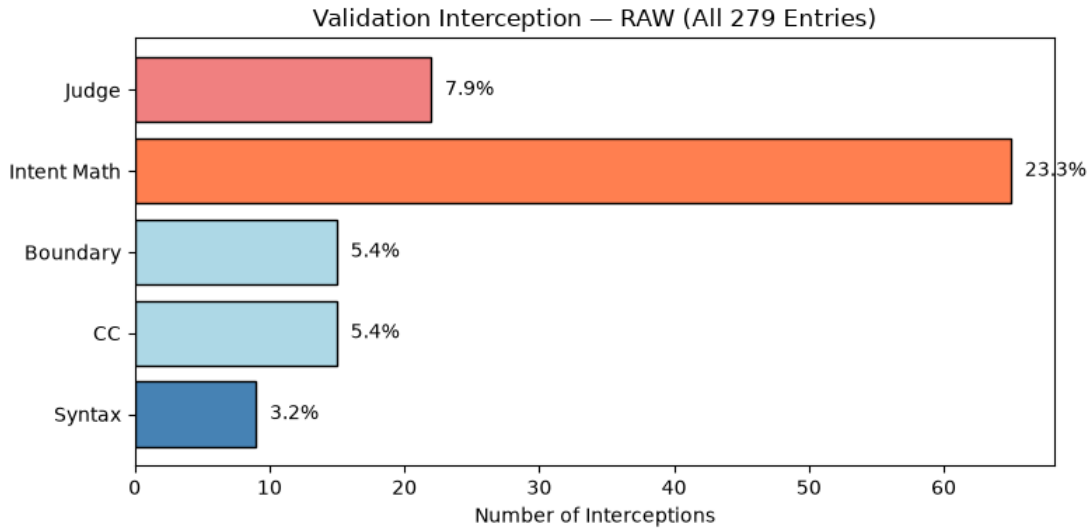
This section analyzes the architectural parameter distribution, deterministic validation interception rate, and iterative feedback resolution rate of the multi-agent system.



Multi-Agent Parameter Distribution

Figure 1. Parameter and Agent Role Distribution: Single-Model Baseline vs. Multi-Agent Framework

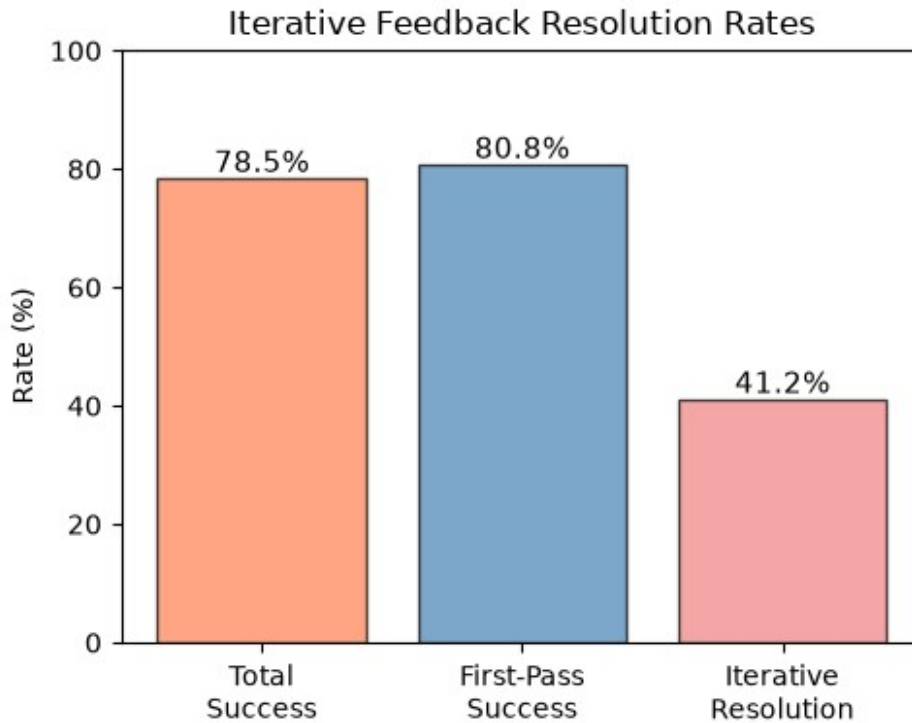
In this figure, it displays the architectural parameter breakdown and model weight allocation comparing the monolithic single-model baseline (Qwen 2.5 7B) against the multi-agent framework. The single-model baseline relies on a single monolithic agent, whereas the multi-agent framework decomposes refactoring into three specialized logical agent roles: Planner, Generator, and Judge. Furthermore, the single-model baseline loads 7 Billion parameters into GPU VRAM at once (Qwen2.5-Coder-7B-Instruct.gguf). In contrast, the multi-agent framework distributes responsibility across two 3B model weight sets (Qwen2.5-Coder-3B-Instruct for Planning and Generation, and Llama-3.2-3B-Instruct for Evaluation/Adjudication), yielding 6 Billion total active parameters across the three logical agent roles.



Deterministic Validation Interception Rate

Figure 2. Deterministic Validation Interception Volume and Gate Results

In this figure, it shows the interception rate and volume of broken or flawed code candidates captured by the multi-tier deterministic validation pipeline across the benchmark dataset ($n=279$). The validation layer serves as an automated quality gate between LLM generation passes and final execution.



Resolution Rate Summary

Figure 3. Iterative Feedback Resolution Rate across Feedback Cycles

In this figure, it demonstrates the ability of the Syntax and Strategy agents to systematically repair intercepted code errors across iterative feedback loops. Overall, the multi-agent system achieved a System Resolution rate of 78.5%, successfully generating or recovering 219 out of 279 tasks.

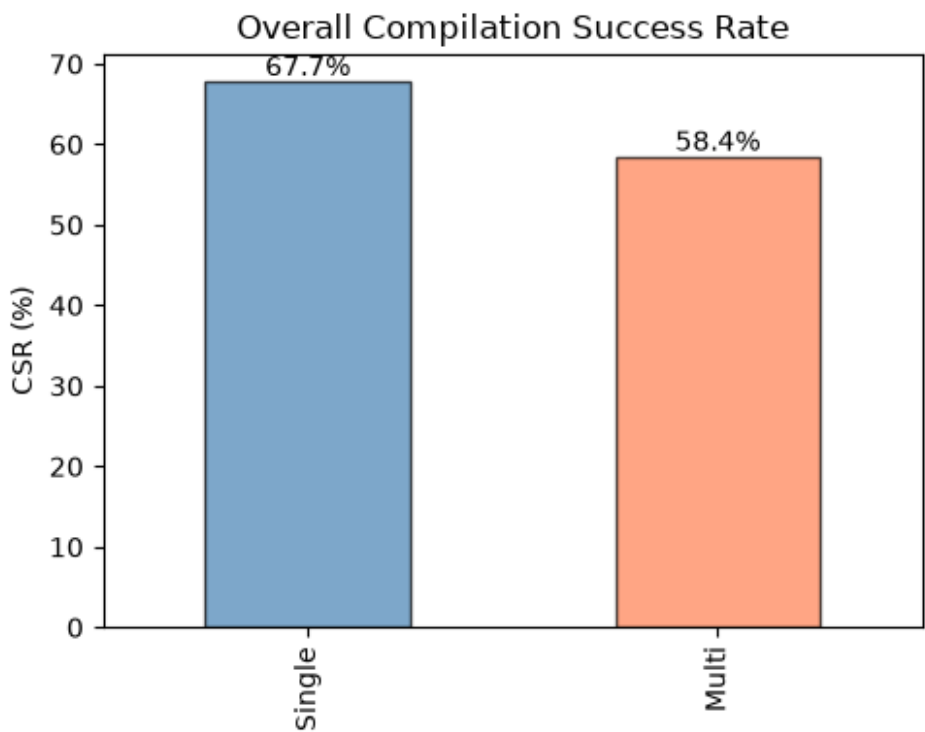
Metric	Single-Model Baseline	Multi-Agent Framework	Difference
System Resolution Rate	N/A	78.5% (219/279)	-
First-Pass Success	N/A	80.8% (177/219)	-

2. Refactoring Performance Evaluation

This section analyzes the compilation success rate, behavioral equivalence rate, cyclomatic complexity, and maintainability index between the Standard single-model approach and the Improved multi-agent framework.

Methodology Note (Funnel Methodology):

To maintain analytical consistency and avoid survivorship bias, the analysis in this section follows a strict funnel methodology. First, the Compilation Success Rate (CSR) is computed across the entire raw dataset ($n=279$) to establish the absolute baseline of syntactically viable code generated by both paradigms. Second, the Logic and Quality Metrics (BER, CC, MI) are evaluated exclusively on Resolved Entries—the subset of code outputs that successfully compiled ($n=189$ for Single-Model Baseline, $n=163$ for Multi-Agent System). This ensures that non-compiling artifacts do not distort logical correctness or software complexity metrics.

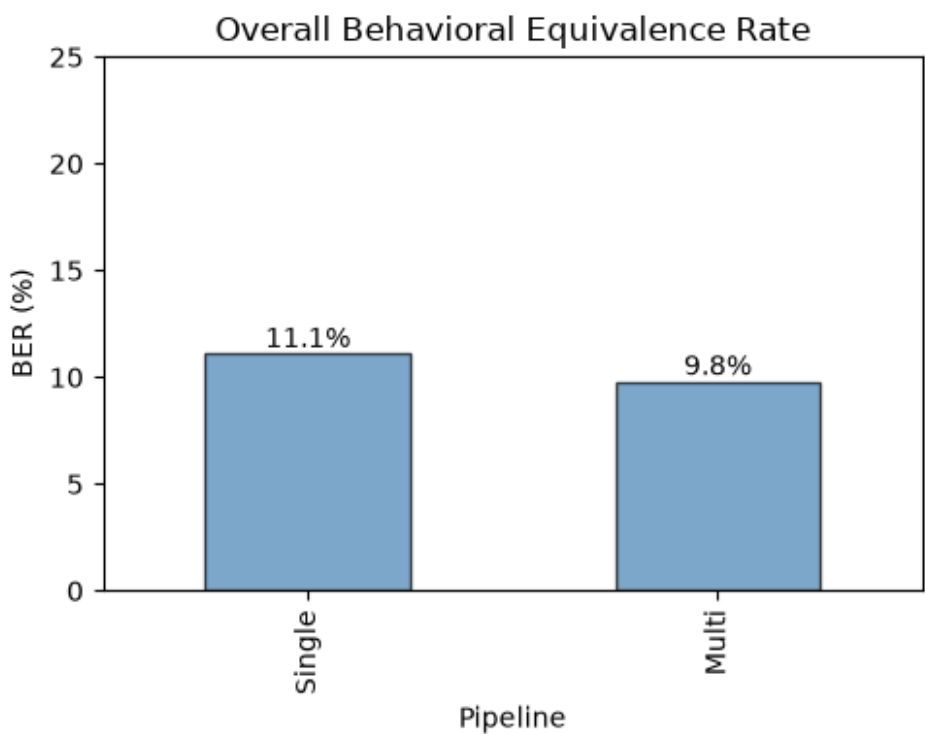


Compilation Success Rate (CSR)

Figure 4. Comparative Compilation Success Rate (CSR) on Raw Dataset ($n=279$)

In this figure, it visually contrasts the raw compilation success rate between the monolithic single-model baseline (Qwen 2.5 7B) and the multi-agent framework

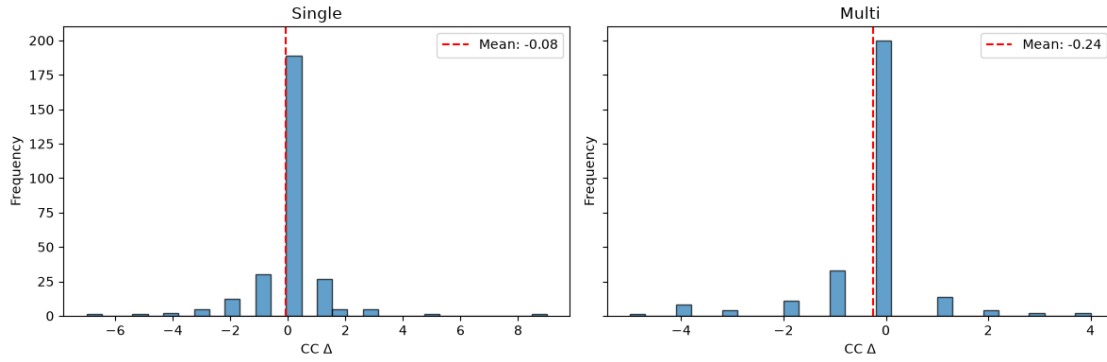
across all 279 benchmark tasks. The single-model baseline Qwen 2.5 7B achieved a raw compilation success rate of 67.7%, successfully compiling 189 out of 279 tasks. In contrast, the multi-agent system achieved a compilation success rate of 58.4%, successfully compiling 163 out of 279 tasks.



Behavioral Equivalence Rate (BER)

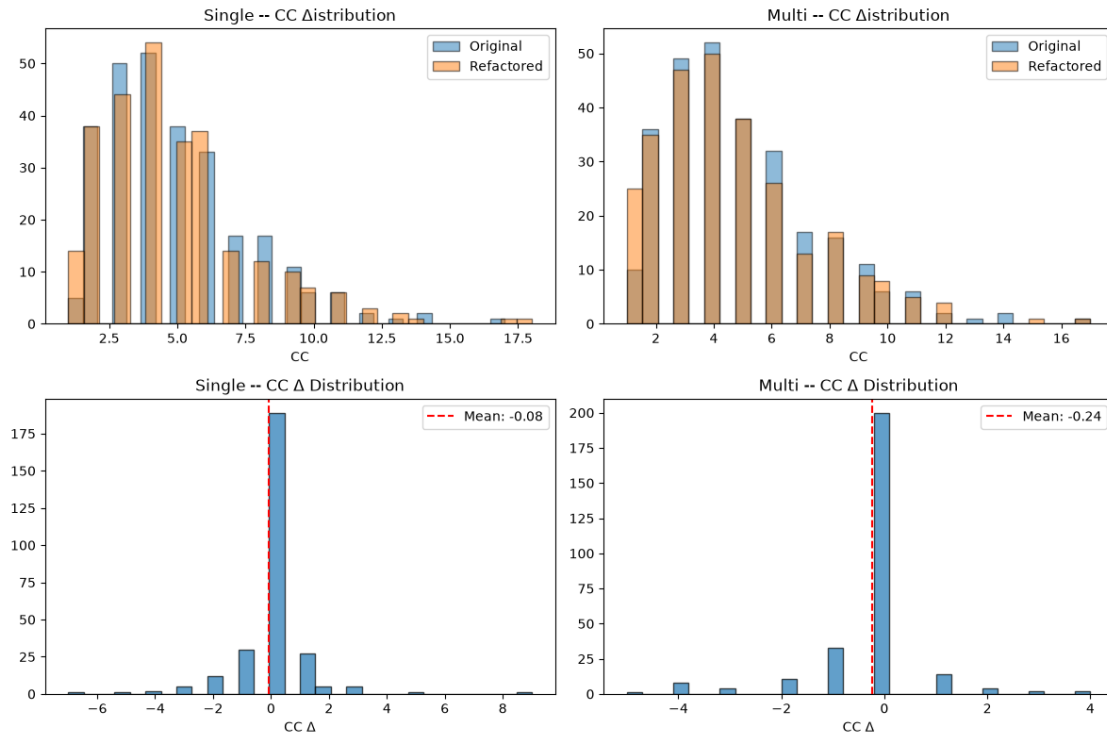
Figure 5. Comparative Behavioral Equivalence Rate (BER) on Compiled Outputs

In this figure, it displays the logical correctness and functional test suite passage among the syntactically valid compiled code outputs. The single-model baseline delivered functional correctness of 11.1%, passing 21 out of 189 compiled tasks. Meanwhile, the multi-agent system achieved a Behavioral Equivalence Rate of 9.8%, successfully passing 16 out of 163 compiled tasks.



Cyclomatic Complexity Overall

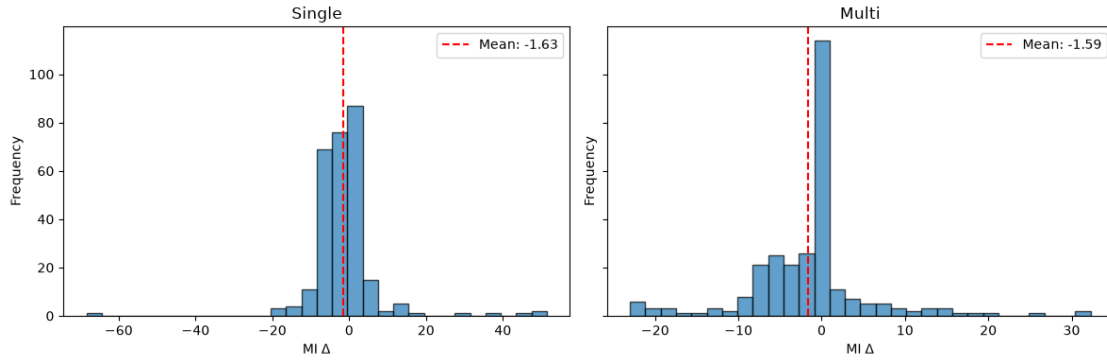
Figure 6. Overall Mean Cyclomatic Complexity Comparison



Cyclomatic Complexity Distribution

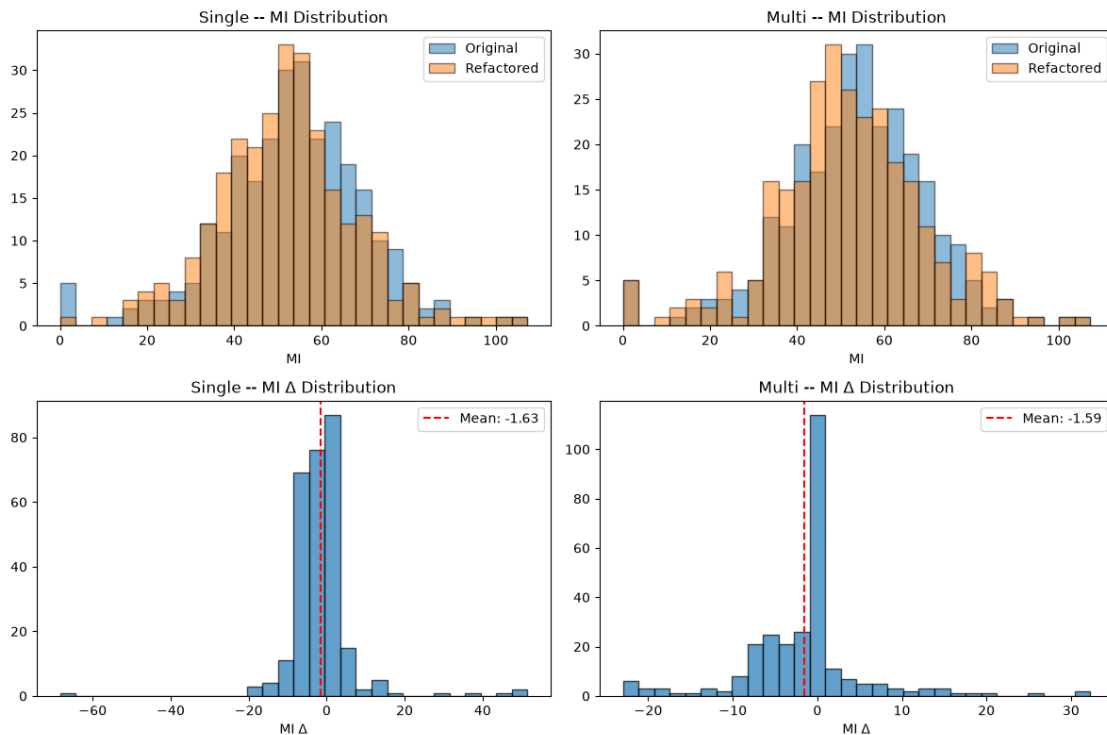
Figure 7. Cyclomatic Complexity Distribution and Density Plot

In these figures, it displays the average reduction and statistical distribution of Cyclomatic Complexity, which measures the number of linearly independent paths through a program's source code. The single-model baseline achieved a Mean Cyclomatic Complexity of 4.90. The multi-agent system achieved a Mean Cyclomatic Complexity of 4.53, showing a measurable reduction in decision branching (lower is better).



Maintainability Index Overall

Figure 8. Overall Mean Maintainability Index Comparison



Maintainability Index Distribution

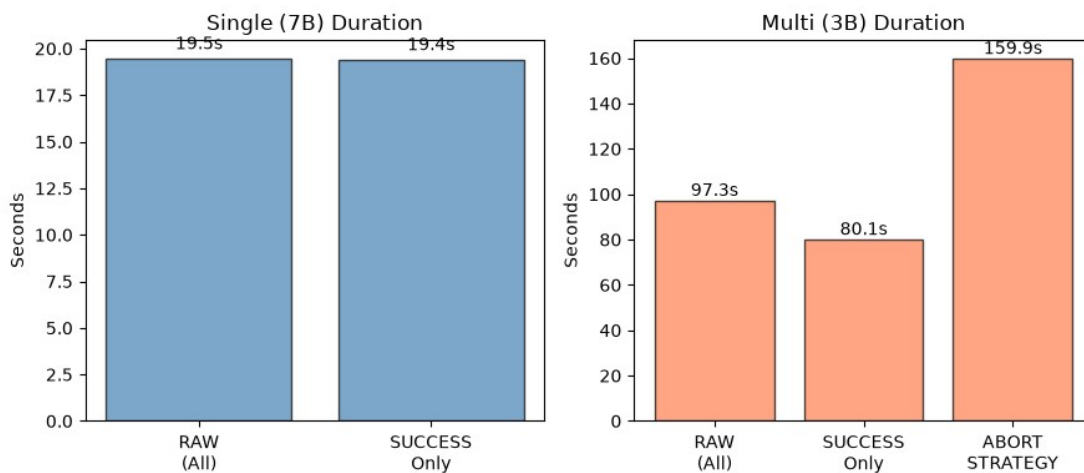
Figure 9. Maintainability Index Distribution Shift

In these figures, it displays the composite Maintainability Index scores. The single-model baseline produced an average Maintainability Index of 51.88. Conversely, the multi-agent system achieved an average Maintainability Index of 53.99, yielding an improvement (higher is better).

Refactoring Metric (Compiled Outputs)	Single-Model Baseline	Multi-Agent Framework	Difference / Outcome
Compilation Success Rate (CSR)	67.7% (189/279)	58.4% (163/279)	Multi lost by 9.3%
Behavioral Equivalence Rate (BER)	11.1% (21/189)	9.8% (16/163)	Multi lost by 1.3%
Cyclomatic Complexity (CC)	4.90	4.53	Multi won (-0.37)
Maintainability Index (MI)	51.88	53.99	Multi won (+2.11)

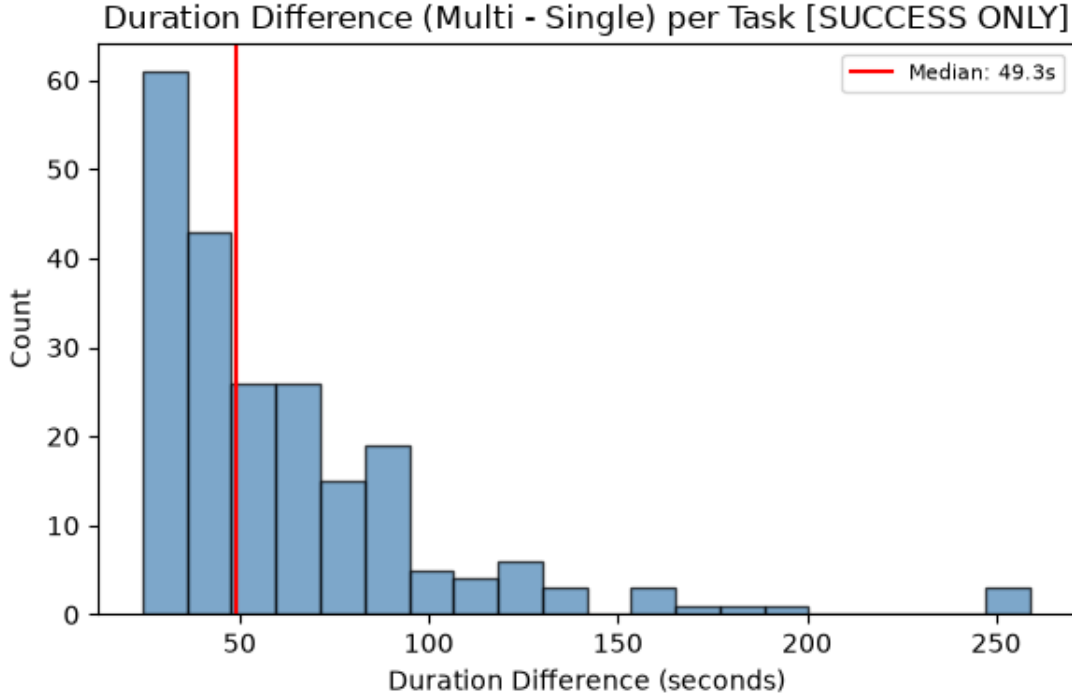
3. Computational Efficiency and Hardware Resource Utilization

This section analyzes the comparison of inference time and peak GPU memory usage between the baseline and the proposed multi-agent framework.



Inference Time Comparison

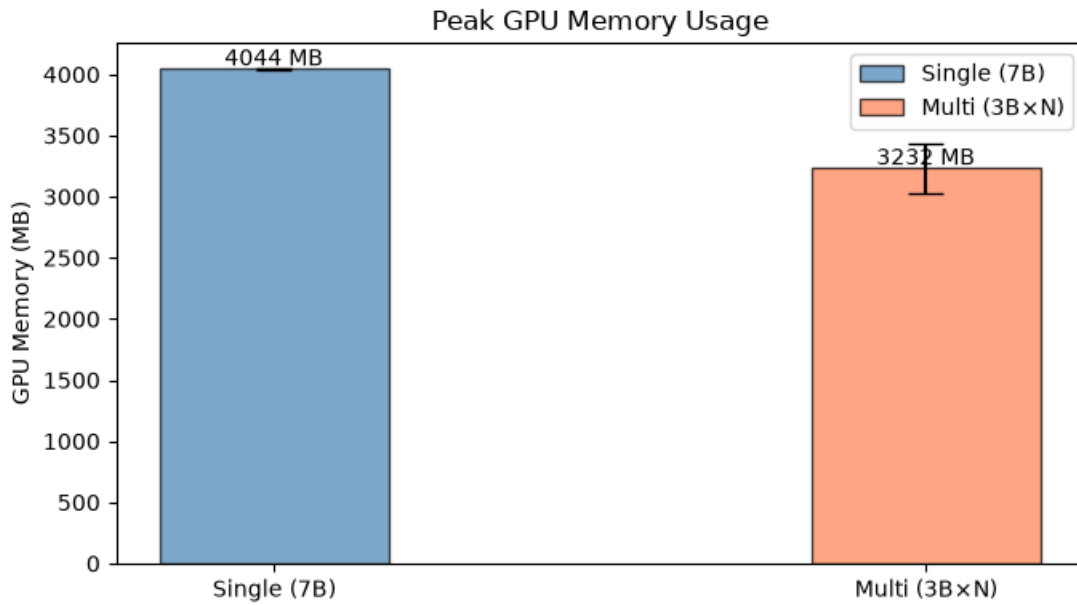
Figure 10. Mean Inference Time Comparison (Seconds per Task)



Duration Difference Wilcoxon Test

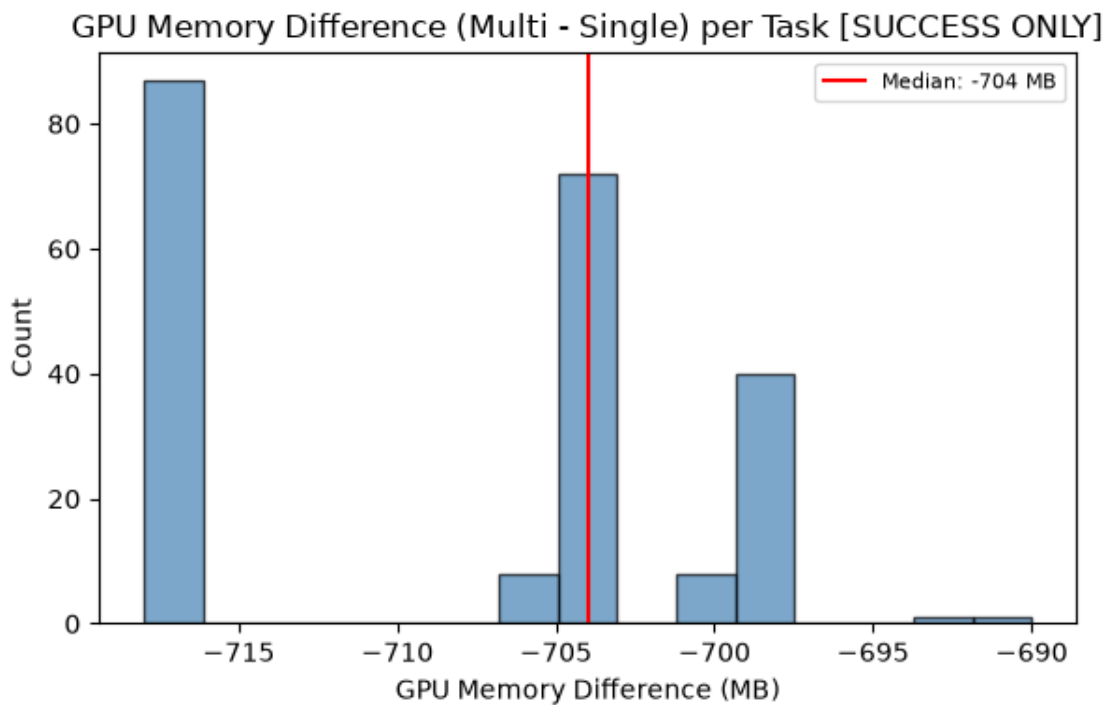
Figure 11. Wilcoxon Paired Difference Distribution for Execution Duration (Δ Duration)

In these figures, it presents the execution latency comparison. The single-model baseline had an average execution time of 19.47 seconds per refactoring task. The multi-agent system had an average execution time of 97.26 seconds per refactoring task, which is an approximate 5x latency increase due to the multi-agent iterative loop. For the statistical significance analysis focusing on normality and paired differences, statistical testing was conducted on paired execution metrics across all tasks ($n=279$ pairs) to capture the true operational overhead of the system. Using the Shapiro-Wilk Test for Normality on duration differences, the result was $W=0.788$ with $p<0.001$. In conclusion, the distribution of duration differences violates normality ($p<0.05$). Consequently, parametric t-tests are invalid, requiring the non-parametric Wilcoxon Signed-Rank Test. Through the Wilcoxon Signed-Rank Test, the statistic was $W=0.0$ with $p<0.001$. This concludes that the overall execution time overhead of the multi-agent system is statistically significant ($p<0.001$).



Peak GPU Memory Comparison

Figure 12. Peak GPU VRAM Memory Allocation Comparison (MB)



GPU Memory Difference Wilcoxon Test

Figure 13. Wilcoxon Paired Difference Distribution for GPU Memory (Δ GPU Memory)

In these figures, it presents the peak GPU memory allocation comparison and paired statistical analysis between the monolithic 7B baseline and the multi-agent 3B model pipeline. The single-model monolithic 7B baseline required an average peak GPU memory allocation of 4,044 MB VRAM. The multi-agent system utilizing 3B sub-models required an average peak GPU memory allocation of 3,232 MB VRAM, representing a 20.1% reduction in peak VRAM consumption, which saves approximately 812 MB.

For the statistical significance analysis regarding normality and paired differences, using the Shapiro-Wilk Test for Normality on GPU memory differences yielded a result of $W=0.530$ with $p<0.001$. In conclusion, the distribution of memory differences is non-normal ($p<0.05$), confirming the requirement for non-parametric evaluation. Through the Wilcoxon Signed-Rank Test, the statistic was $W=0.0$ with $p<0.001$. This concludes that the overall reduction in peak VRAM usage achieved by the multi-agent framework is statistically significant ($p<0.001$).

Efficiency Metric	Single-Model Baseline	Multi-Agent Framework	Difference
Mean Execution Time (Duration)	19.47 s / task	97.26 s / task	Multi is ~5x slower
Peak GPU VRAM Allocation	4,044 MB	3,232 MB	Multi uses 812 MB less

Statistical Significance Results (All Tasks, $n=279$)

The table below summarizes the formal statistical testing for differences in execution duration and peak GPU memory between the monolithic baseline and the multi-agent framework across all tasks. Because the Shapiro-Wilk test confirmed that the paired differences are not normally distributed ($p<0.05$), the non-parametric Wilcoxon Signed-Rank test was required. Both the overall latency increase and the overall VRAM reduction achieved by the multi-agent framework were found to be highly statistically significant.

Metric (Difference: Multi - Single)	Shapiro-Wilk Normality Test (W , p -value)	Wilcoxon Signed-Rank Test (W , p -value)	Outcome
Execution Duration	$W=0.788$, $p=9.18 \times 10^{-19}$	$W=0.0$, $p=1.66 \times 10^{-47}$	Statistically Significant ($p<0.05$)

Metric (Difference: Multi - Single)	Shapiro-Wilk Normality Test (W , p -value)	Wilcoxon Signed- Rank Test (W , p - value)	Outcome
	(Non-normal)		)
Peak GPU Memory	$W = 0.530$, $p = 1.15 \times 10^{-26}$ (Non-normal)	$W = 0.0$, $p = 3.95 \times 10^{-48}$	Statistically Significant ($p < 0.05$)